Experiment 8

Student Name: Arpit Anand **UID: 23BCS12710**
Branch: CSE **Section/Group: KRG 1B**
Semester: 6th **Date of Performance: 06/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable **Stock Trading Platform** (like Zerodha/Groww/Upstox) that allows users to trade stocks in real-time, track market prices, manage portfolios, and perform transactions efficiently using distributed system architecture.

2. **Objective**:

- To identify functional and non-functional requirements of a stock trading system
- To design High Level Architecture (HLD) for trading platforms
- To understand real-time stock price streaming
- To design microservices for trading, portfolio, and payments
- To study order execution and validation mechanisms
- To understand scalability challenges in financial systems

3. **Tools:**

- Draw.io / Excalidraw – System design diagrams
- MySQL / PostgreSQL – User & transaction data
- Redis – Caching & real-time data
- Kafka – Event streaming (stock prices/orders)
- Load Balancer – Traffic distribution
- WebSockets – Real-time stock updates
- Microservices Architecture
- InfluxDB – Time-series stock data

4. **System Requirements:**

A. Functional Requirements:-

- Users should be able to register, login, and complete KYC
- Users should view real-time stock prices and historical data
- Users should place buy/sell orders
- Users should modify or cancel orders
- System should provide a watchlist with live updates

- Users should view portfolio (holdings, P&L, history)
- Users should deposit/withdraw funds
- System should validate orders before execution
- System should show order status in real-time

B. Non-Functional Requirements:-

1) Scalability

a. Support 2–3 million daily active users

b. Handle 8k–10k stocks

2) Consistency (CAP Theorem)

a. Consistency > Availability

b. No duplicate trades / wrong balances

3) Availability

a. Real-time stock prices should always be available

4) Latency

a. Order placement < 100 ms

b. Market data < 50 ms

5) Reliability

a. No loss of trade or order data

5. High Level Design (HLD):

System follows Microservices Architecture

Main Components:

1) API Gateway

a. Authentication

b. Authorization

c. Routing

d. Rate Limiting

2) **User Service**
    a. Registration & login
    b. KYC verification
    c. User DB (MySQL)
3) **Price Tracker Service**
    a. Fetches real-time stock data from exchange (NSE/BSE)
    b. Uses Kafka + Redis for streaming
4) **Order Service**
    a. Handles buy/sell orders
    b. Stores order data
5) **Validation Service**
    a. Checks KYC
    b. Checks funds availability
    c. Prevents invalid trades
6) **Portfolio Service**
    a. Tracks holdings
    b. Calculates P&L
7) **Watchlist Service**
    a. Stores user watchlist
    b. Provides real-time updates
8) **Payment Service**
    a. Deposit / Withdraw funds
9) **Exchange Server**
    a. External system (NSE/BSE) for trade execution

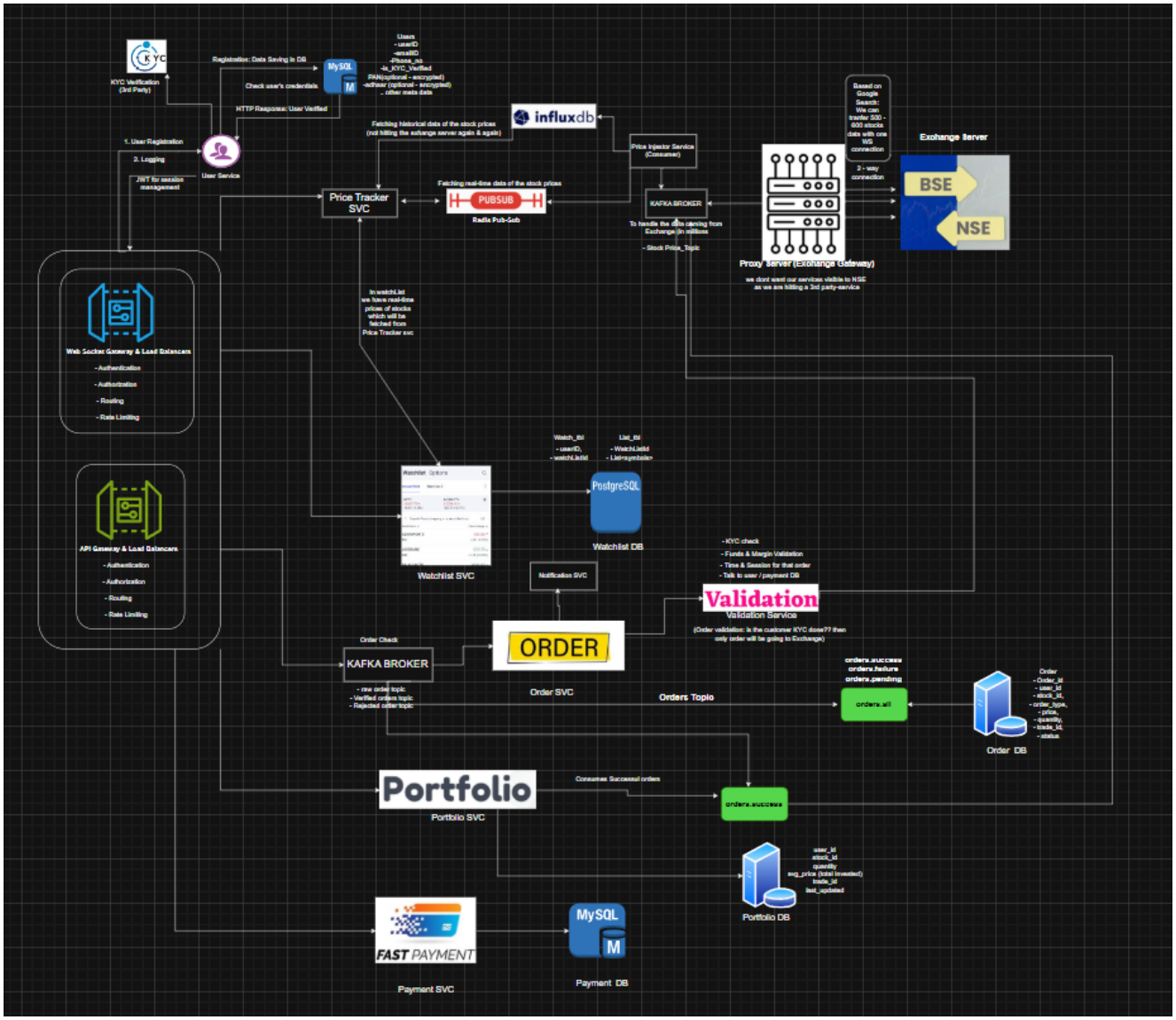
6. Trading Flow (Step-by-Step):

1) User logs in and completes KYC
2) User views stock prices (real-time via WebSockets)
3) User adds stocks to watchlist
4) User places buy/sell order
5) Order goes to Validation Service
6) If valid → sent to Exchange Server
7) Order executed
8) Order status updated
9) Portfolio updated
10) User sees updated P&L

7. Scalability Solution:

To support millions of users:

- Use Kafka for real-time stock streaming
- Use Redis for caching stock prices
- Use WebSockets for live updates
- Use Load Balancers for traffic distribution
- Use Microservices architecture
- Use separate databases (User, Order, Portfolio, Payment)
- Use InfluxDB for time-series stock data
- Use API Gateway for centralized control



8. Learning Outcomes (What I Have Learnt):

- Learned how stock trading platforms are designed
- Understood real-time data streaming using Kafka
- Learned order validation and execution flow
- Understood portfolio management systems
- Learned importance of consistency in financial systems
- Understood microservices and scalability
- Learned how exchanges (NSE/BSE) integrate with apps